OKV TECHNOLOGY CONSULTS
Nigeria Payroll & HR Management System

FULL DOCUMENTATION

Full Technical Documentation

Architecture, data model, API, security, deployment, operations and roadmap for the OKV Nigeria Payroll & HR Management System

SEPTEMBER 2026 EDITION

technologyokv@gmail.com | +234 810 414 1138 | www.okvtechnology.com

© 2026 OKV Technology Consults. All rights reserved.

Contents

1. System overview 4

1.1 Design goals 4

1.2 Technology stack 4

1.3 Logical architecture 5

2. Repository inventory 6

3. Data model 7

3.1 Master spreadsheet 7

3.2 Tenant spreadsheet (one per organisation) 7

3.3 Business entities (the Data tab) 8

4. Backend API reference 9

5. Front-end architecture 11

5.1 Pages and how they connect 11

5.2 The SPA shell 11

5.3 Calculation engine 11

5.4 Local storage and session 12

5.5 Synchronisation 12

5.6 Service worker 12

5.7 Demo mode 12

6. Security and privacy model 13

6.1 What is implemented 13

6.2 Findings and hardening recommendations 13

7. Key business workflows 15

7.1 Sign-up and trial 15

7.2 Upgrade by bank transfer 15

7.3 Reminder emails 15

7.4 Capacity monitoring 15

7.5 Suspension 15

8. Deployment guide 16

8.1 Prerequisites 16

8.2 Backend . . . 16
8.3 Front end . . . 16
8.4 Configuration reference . . . 17

9. Operations and maintenance . . . 18
9.1 Routine tasks . . . 18
9.2 Platform quotas to plan for . . . 18
9.3 Incident playbook . . . 19

10. Testing . . . 20
10.1 Automated checks run for this release . . . 20
10.2 Release smoke test (10 minutes) . . . 20

11. Known limitations and roadmap . . . 21

12. Change log for this release . . . 22

Appendix — Quick reference . . . 23
Apps Script one-time functions . . . 23
Role table (client and server) . . . 23
Contacts . . . 23

1. System overview

The **OKV Nigeria Payroll & HR Management System** is a multi-tenant software-as-a-service product. Each customer organisation gets an isolated workspace for employee records, payroll, payslips, leave, loans, communications and reports. It is delivered as a static web front end (a Progressive Web App) talking to a Google Apps Script backend that stores data in Google Sheets. There is no separate database server, no container, and no monthly infrastructure bill beyond a static host.

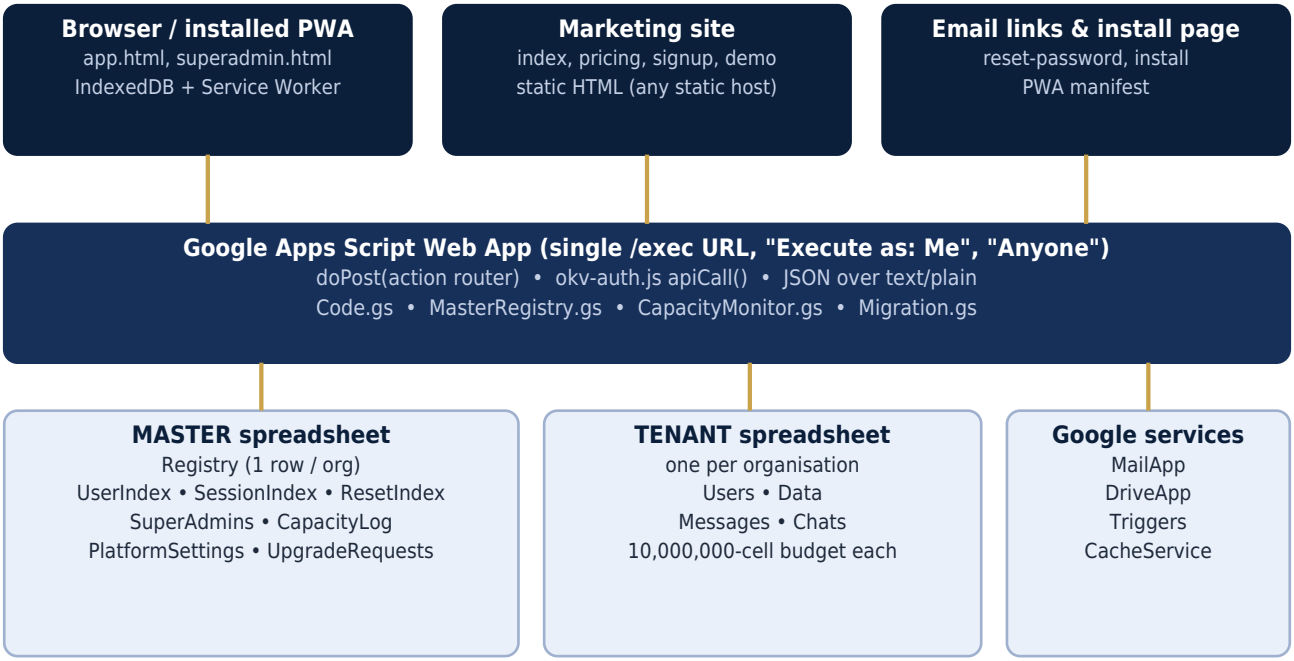
1.1 Design goals

Goal	How it is achieved
Works with unreliable power and networks	Offline-first PWA: every change is written to IndexedDB immediately; a service worker caches the app shell; changes sync when a connection exists.
Near-zero running cost and simple operations	Google Apps Script + Google Sheets as the backend; static hosting for the front end; email through MailApp; scheduled work through Apps Script triggers.
Tenant isolation	One spreadsheet per organisation (*per-tenant* architecture) resolved through a Master registry; every API call is scoped by the caller's `orgId`.
Nigerian statutory payroll	Client-side calculation engine for PAYE, pension, NHF and life assurance with editable rates (see the 2026 tax-law caveat in section 11).
Low-friction onboarding	7-day free trial with no payment step; manual bank-transfer upgrades approved by editing a spreadsheet cell.

1.2 Technology stack

Layer	Technology
Front end	Hand-written HTML/CSS/JavaScript (no build step), Bootstrap 5.3.3, Bootstrap Icons 1.11.3, Chart.js 4.4.4 — loaded from jsDelivr and cached by the service worker.
Client storage	IndexedDB (per organisation, per device) and `localStorage` for the session token.
Offline / install	Service worker (`app/sw.js`), Web App Manifest, install page with per-platform instructions.
Backend	Google Apps Script (V8) deployed as a Web App; actions dispatched by a single `doPost` router.
Storage	Google Sheets: one Master spreadsheet plus one spreadsheet per organisation; Google Drive for payment-screenshot files.
Email	Apps Script `MailApp` (welcome, reset, reminders, upgrade decisions, announcements, payslips).
Scheduling	Time-driven triggers (subscription reminders, capacity monitor) and an on-edit trigger (upgrade approvals).
Testing used in this release	Playwright + headless Chromium with a mock API, exercised across all roles (section 10).

1.3 Logical architecture



Everything the browser sends goes to one Apps Script URL. The script authenticates the session, finds the caller's organisation in the Master *Registry*, opens that organisation's own spreadsheet, and reads or writes only rows belonging to it.

2. Repository inventory

File	Role
`Code.gs`	Core backend: configuration constants, action router, auth/session, users, data sync, announcements, internal chat, upgrade requests, platform settings, Super Admin actions, reminder emails.
`MasterRegistry.gs`	Master-spreadsheet layer: Registry / UserIndex / SessionIndex / ResetIndex / SuperAdmins / CapacityLog, tenant spreadsheet creation and lookup (with script cache).
`CapacityMonitor.gs`	Daily per-tenant cell-usage measurement, growth projection, status flags and Super Admin alert email.
`Migration.gs`	One-off migration from the old single-spreadsheet layout to per-tenant spreadsheets, plus cleanup.
`index.html, pricing.html, signup.html`	Marketing site, plan picker (live prices from the platform), sign-up form.
`demo.html`	Public read-only sandbox with seeded data and an in-browser mock API; includes a “View as” role switcher.
`privacy-policy.html, terms-of-service.html, refund-policy.html`	Legal pages (NDPA 2023-aligned privacy policy; 30-day money-back guarantee).
`okv-auth.js` (root) and `app/okv-auth.js`	Shared client helper: `API_URL`, `apiCall()`, SHA-256 hashing, session storage. **Two identical copies** (service-worker scope).
`app/index.html`	Login page; routes Super Admins to `superadmin.html`.
`app/app.html`	The organisation dashboard (single-file SPA): all 21 tabs, calculation engine, IndexedDB layer, sync, role enforcement.
`app/superadmin.html`	Platform dashboard: organisations, capacity, announcements, branding, payments, comms, pricing, account.
`app/install.html, reset-password.html`	Install instructions by platform; password-reset form.
`app/sw.js, manifest.json, icons/`	Service worker (cache `okv-payroll-online-v4`), manifest, icons.
`app/manuals/*.pdf`	Downloadable User Manual (Full) and Quick Guide (Self-Service), served by the User Manual tab.
`robots.txt, sitemap.xml`	SEO files (update the placeholder site URL before launch).
`README.md`	Deployment steps, sheet headers, testing walkthrough.

3. Data model

3.1 Master spreadsheet

The spreadsheet that holds the Apps Script project is the **Master**. It stores platform-wide data and the index that routes each request to the right tenant.

Tab	Columns	Purpose
Registry	orgId, orgName, spreadsheetId, adminUserId, adminFullName, adminEmail, adminPhone, plan, billingCycle, status, subscriptionStatus, subscriptionStartDate, subscriptionExpiry, trialEndsAt, createdAt, updatedAt, lastReminderSentAt, cellsUsed, cellPercentUsed, rowsAddedRecent, estMonthsRemaining, capacityStatus, lastCapacityCheck, capacityAlertSent	**Authoritative** organisation record: lifecycle, plan, trial/subscription dates, tenant spreadsheet ID and capacity metrics.
UserIndex	username, email, orgId	Global lookup: which organisation does this login belong to?
SessionIndex	sessionToken, orgId, userId, expiry	Maps a bearer token to organisation and user without scanning tenants.
ResetIndex	resetToken, orgId, userId, expiry	Maps a password-reset token to its owner.
SuperAdmins	userId, fullName, username, email, passwordHash, status, sessionToken, sessionTokenExpiry, resetToken, resetTokenExpiry, createdAt, updatedAt	Platform-level accounts (not tied to an organisation).
CapacityLog	orgId, timestamp, cellsUsed	Daily snapshots used to project growth.
PlatformSettings	key, value (one row, key = “config”, JSON)	Branding, contact details, payment methods, gateway and SMS/WhatsApp credentials, plan prices.
UpgradeRequests	orgId, requestId, userId, orgName, fullName, email, phone, planRequested, billingCycle, amount, screenshotUrl, status, submittedAt, decidedAt, subscriptionStartDate, subscriptionEndDate	Manual bank-transfer upgrade queue; editing *status* triggers approval.
(platform messages)	messageId, fromUserId, fromName, toScope, toLabel, channel, …	Log of Super Admin broadcasts.

3.2 Tenant spreadsheet (one per organisation)

Created automatically at sign-up and named `OKV Payroll — <Organisation> (<orgId>)`. An optional Drive folder can hold them all (`TENANT_FILES_FOLDER_ID`).

Tab	Columns	Purpose
Users	orgId, orgName, userId, fullName, username, email, phone, passwordHash, isAdmin, roles, plan, billingCycle, status, subscriptionStatus, subscriptionStartDate, subscriptionExpiry, trialEndsAt, resetToken, resetTokenExpiry, sessionToken, sessionTokenExpiry, employeeId, createdAt, updatedAt, isSuperAdmin, lastReminderSentAt	Users of this organisation. *roles* is a comma-separated list; *employeeId* links a user to an employee record for My Portal.

Tab	Columns	Purpose
Data	orgId, entityType, recordId, jsonData, updatedAt, updatedBy	All business data as JSON rows, one row per record.
Messages	orgId, messageId, fromUserId, fromName, toScope, toLabel, channel, messageType, subject, body, recipientCount, sentAt	Log of announcements and payslip emails sent.
Chats	orgId, chatId, fromUserId, toUserId, body, sentAt, readAt	Internal Admin↔User messages; *chatId* is the two user IDs sorted and joined.

3.3 Business entities (the Data tab)

The `Data` tab stores seven entity types. The front end holds each as a JavaScript collection and as an IndexedDB store.

entityType	Record key	Fields (JSON)
employees	id	id, surname, first, dept, desig, grade, bank, acct, basic, housing, transport, other, pension (Yes/No), status, dateEmployed, email, phone
payrollInputs	empId	overtime, bonus, loan, remarks (current period)
leaveRegister	id	id, empId, name, dept, type, start, end, status
loanRegister	id	id, empId, name, dept, amount, monthly, balance, status
bioData	empId	dob, gender, marital, state, qualification, nokName, nokRelation, nokPhone
settings	“main”	companyName, companyAddress, companyPhone, companyEmail, rcNumber, companyLogo, payrollMonth, payrollYear, pensionEmpRate, pensionErRate, nhfRate, lifeRate, craFlat, craPct1, craPct2, workDays, minWage
payrollHistory	id (empId_YYYY-MM)	Snapshot per employee per confirmed month: identity, bank, salary components, gross, overtime, bonus, pension, NHF, CRA, PAYE, loan, netPay, rates used, archivedAt

Why JSON rows?

Storing entities as JSON in a fixed six-column tab means new fields need no schema migration, every tenant has an identical, tiny sheet structure, and the front end stays the single source of truth for business rules.

4. Backend API reference

All calls are POST requests to the Web App URL with a JSON body sent as `text/plain;charset=utf-8` (this avoids a CORS pre-flight, which Apps Script does not answer). Every body has an `action`. Every response is JSON with `success: true|false` and, on failure, an `error` message that the client shows to the user. A GET returns a simple health message.

```
POST <API_URL>
Content-Type: text/plain;charset=utf-8

{ "action": "getData", "sessionToken": "tok_..." }

200  { "success": true, "entities": { "employees": [ {"recordId":"EMP001","data":{...}} ], ... } }
```

Authorisation levels. *Public* = no token. *Session* = any valid, unexpired session of an active user in an active organisation with a valid trial or subscription. *Comms* = session plus a role in `COMMS_ROLES`. *Admin* = the organisation Admin. *Super* = platform Super Admin.

Action	Auth	Purpose and notes
signup	Public	Creates the organisation: tenant spreadsheet, Registry row, Admin user, indexes, 7-day trial, welcome email; returns a session.
login / superAdminLogin	Public	Verifies username + SHA-256 password hash; checks user/organisation status and trial/subscription; issues a 30-day session token.
validateSession	Session	Returns the current user (with live organisation subscription state).
forgotPassword	Public	Admin accounts only; emails a single-use link (60 min). Replies generically to avoid account enumeration.
resetPassword	Public	Sets a new password hash from a valid reset token.
changePassword / superAdminChangePassword	Session	Requires the current password hash.
adminListUsers / adminCreateUser / adminUpdateUserRoles / adminSetUserStatus / adminSetUserPassword / adminDeleteUser	Admin	User management scoped to the Admin's organisation.
getData	Session	Returns every entity type for the caller's organisation.
pushEntity	Session	**Replaces** all rows of one entity type for the organisation with the supplied records. Rejects unknown entity types.
sendMessage	Comms	Sends email / SMS / WhatsApp to a recipient list (personalised bodies allowed) and logs it; returns *sent* and a *failed* list with reasons.
sendInternalMessage / getInternalMessages / listChatThreads	Session	Admin↔User chat, unread counts and thread list.
submitUpgradeRequest	Admin	Uploads the payment screenshot to Drive, appends an UpgradeRequests row, emails the company.
getUpgradeRequests	Admin	The organisation's own request history.

Action	Auth	Purpose and notes
getPublicPlatformInfo	Public	Live plans, enabled payment methods and gateway flags (no secrets) for the pricing page and upgrade form.
superAdminListOrgs / UpdateOrgContact / SetOrgStatus / SetOrgPlan	Super	Organisation administration (suspend, reactivate, edit contact and plan/dates).
superAdminSendMessage	Super	Broadcast to all or selected organisations.
superAdminGetSettings / SaveSettings	Super	Read and write the PlatformSettings JSON.
superAdminGetCapacityOverview	Super	Cells used, percentage, projected time remaining and status per organisation.

Session validation order

`requireSession_` checks, in order: token exists (via SessionIndex) → not expired → user Active → organisation Active → trial not ended / subscription Active and not past expiry. The Registry is authoritative for subscription state, so a trial ending or an organisation being suspended takes effect on the very next API call.

5. Front-end architecture

5.1 Pages and how they connect

Page	Loads	Notes
index.html (root)	okv-auth.js	Marketing landing page; links to pricing, demo and login.
pricing.html	okv-auth.js	Default prices are embedded; `getPublicPlatformInfo` overrides them live.
signup.html	okv-auth.js	Posts `signup` with a SHA-256 password hash; saves the session; redirects to install page.
app/index.html	app/okv-auth.js	Login. Super Admin sessions are routed to superadmin.html.
app/app.html	app/okv-auth.js	The organisation SPA (21 tabs). Redirects to login when no valid session exists.
app/superadmin.html	app/okv-auth.js	Platform dashboard; requires a Super Admin session.
demo.html	okv-auth.js (unused)	Sandbox: seeded data, in-browser mock API, mutations locked, role switcher.

5.2 The SPA shell

- **Navigation.** Sections are `<section id="sec-NAME">` blocks; sidebar links carry `data-section`. `showSection()` toggles the active section, updates the title and calls the matching render function from a `renderSection` table.
- **State.** Module-level variables: `employees, payrollInputs, leaveRegister, loanRegister, bioData, payrollHistory, settings, currentUser, sessionToken`. Renders read these directly; there is no framework.
- **Role enforcement.** `ROLE_SECTIONS` maps each role to tab names; `allowedSections()` unions a user's roles; `applySidebarAccess()` hides links and `showSection()` refuses sections outside the set. The Admin sees all; a user with no roles gets My Portal.
- **Charts.** Chart.js for the dashboard and reports. **Toasts** and Bootstrap modals for feedback and forms.
- **Printing.** Print stylesheets hide chrome; payslips open in a print-ready document generated from `payslipBlockHTML()`.

5.3 Calculation engine

The engine lives in the SPA ("mirrors the Excel formulas 1:1"). Key functions: `grossMonthly, emolument, pensionMonthly, pensionErMonthly, nhfMonthly, lifeAssuranceMonthly, craAnnual, payeAnnualFromTaxable, computePayroll`. Bands are the constant `PAYE_BANDS`; rates come from `settings`. Chapter 6 of the Master Manual documents the formulas and a worked example produced by this code.

> **Calculation runs on the client**
>
> Because payroll is computed in the browser, the server stores whatever the client sends (`pushEntity`) and performs no payroll validation. This is fast and works offline, but means correctness depends on the client code shipped to users.

5.4 Local storage and session

Store	Key	Content
localStorage	okv_session_v1	JSON {user, sessionToken, savedAt} for organisation users.
localStorage	okv_superadmin_session_v1	Same, for the Super Admin.
IndexedDB	okv_payroll_online_<orgId>	Object stores: employees, payrollInputs, leaveRegister, loanRegister, bioData, settings, meta, payrollHistory.
Cache Storage	okv-payroll-online-v4	App shell + CDN libraries for offline start.

5.5 Synchronisation

1. **Write path.** Every edit updates the in-memory collection, writes the IndexedDB store, and queues the store name in `pendingPush`; a debounced (about 1.5 s) `pushEntity` sends the *whole* collection.
2. **Read path.** `syncNow()` first flushes queued pushes, then calls `getData` and replaces local collections with the server's (`pullAllData`).
3. **Triggers.** Manual Sync button; automatic every 30 s while online; immediately on the browser's `online` event; after login.
4. **Status.** *Synced, Pending sync, Syncing, Offline, Sync error* in the top bar; an offline strip appears when `navigator.onLine` is false.
5. **Conflict rule.** Last writer wins per entity type (see limitation L5).

> **Fixed in this release: archived payroll and sync**
>
> The server's allowed entity list omitted `payrollHistory`. Pushes of archived pay runs were rejected, and the next pull returned nothing, which **erased archived months on the device**. `Code.gs` now lists `payrollHistory`, and the client keeps local archives if an older backend omits the list. **Redeploy the script** (new version) to activate the server half.

5.6 Service worker

`app/sw.js` precaches the app shell and CDN assets on install and serves cache-first, falling back to the network; API calls to `script.google.com` and non-GET requests are never intercepted. Activation deletes older caches. **Whenever you change any file under app/, bump** `CACHE_VERSION` so installed copies fetch the update (this release uses `okv-payroll-online-v4`). The service worker's scope is `/app/`, which is why `okv-auth.js` exists twice.

5.7 Demo mode

- Loads fixed sample employees, leave, loans and BioData; no backend calls.
- A local `apiCall()` stand-in serves Messages, Announcements and Upgrade & Subscription so the *real* application code runs unchanged; nothing is stored.
- A "View as" selector switches between Admin and the seven roles by changing `currentUser` and re-applying sidebar access.
- Mutating functions (save, delete, import, restore) are replaced by a sign-up-to-unlock prompt. A "Get Started" card appears after about 12 s.

6. Security and privacy model

6.1 What is implemented

Area	Implementation
Tenant isolation	Separate spreadsheet per organisation; every query filtered by the caller's `orgId` derived from the session, never from the request body.
Authentication	Username + password. The browser hashes the password with SHA-256 before sending; the server stores and compares the hash.
Sessions	Random bearer token, 30-day expiry, stored server-side (SessionIndex) and in the tenant Users row; validated on every call.
Account lifecycle	Active/Suspended at user and organisation level, checked on every call; trial and subscription expiry enforced server-side.
Password reset	Admin-only email flow; token single-use, 60-minute expiry; generic response to prevent account enumeration.
Authorisation (API)	Admin-only actions for user management, upgrades and request history; comms permission for sending; Super-Admin-only platform actions.
Transport	HTTPS (Apps Script and static hosts); requests sent as text/plain to avoid CORS pre-flight.
Privacy documents	NDPA 2023-aligned Privacy Policy positioning each customer as Data Controller and OKV as Processor; Terms; Refund Policy.

6.2 Findings and hardening recommendations

The items below were found while reading the code for this documentation. They do not stop the product from working, but each matters for a system that stores salaries and bank details. They are listed from highest to lowest priority.

ID	Finding	Why it matters	Recommended fix
S1	**Role restrictions are enforced in the interface, not the API.** `getData` and `pushEntity` accept any valid session; every user receives the organisation's full dataset on the device.	A Self-Service user's browser holds all employees' salaries, and a technically capable user with a token could read or overwrite any entity type.	Add role checks to `getData`/`pushEntity` (return only the caller's own records to Self-Service users; allow writes only for roles that own that entity).
S2	**Passwords are hashed with unsalted SHA-256 in the browser.**	The hash is effectively the password for replay; unsalted fast hashes are weak if the sheet is ever exposed.	Move to server-side salted, slow hashing (for example PBKDF2 with per-user salt) and re-hash on next login.
S3	**No login throttling.** The Web App is public ("Anyone").	Online password guessing is unlimited.	Add per-username/IP attempt counters with lockout using CacheService; add CAPTCHA at sign-up.
S4	**Default Super Admin credentials** are created by `setupSuperAdmin()` and appear in source and README.	Anyone with the repository knows the initial password.	Change it at first login (documented); better, read the initial password from Script Properties and delete the constant.

ID	Finding	Why it matters	Recommended fix
S5	**Payment screenshots are shared “anyone with the link”** in Drive.	Receipts can contain account details; links can leak.	Store in a private folder and open by owner only; put the file ID, not a public URL, in the sheet.
S6	**Session token and local data are unencrypted on the device** (localStorage / IndexedDB).	A shared or stolen device exposes payroll data and a live session.	Shorter sessions for shared devices, an idle-timeout lock, and guidance to use device screen locks.
S7	**Broadcast and announcement bodies are sent as HTML.**	Low risk today (authenticated senders), but content is not sanitised.	Sanitise or restrict tags on the server before `MailApp`.

7. Key business workflows

7.1 Sign-up and trial

1. Browser posts `signup` (organisation, admin name, email, phone, username, password hash, plan, cycle).
2. Server rejects duplicate usernames/emails (UserIndex), creates the tenant spreadsheet with its four tabs, appends the Registry row (`status=Active, subscriptionStatus=trialing, trialEndsAt=now+7d`), adds the Admin user with role HR/Payroll Super Admin, indexes the user and session, and emails a welcome message.
3. Client stores the session and shows the install page, then the login page.

7.2 Upgrade by bank transfer

1. Admin transfers the amount to an enabled payment method, then submits plan, cycle and a screenshot.
2. `submitUpgradeRequest` stores the image in Drive, appends a **Pending** row to UpgradeRequests (amount taken from platform pricing) and emails the company address.
3. The operator compares the payment with the bank statement and sets the row's *status* to **Confirmed** or **Rejected**.
4. The installable on-edit trigger `onUpgradeSheetEdit` records *decidedAt*, and for Confirmed: sets start = now and end = now + 30 / 182 / 365 days (monthly / bi-annual / yearly), updates the Registry and the Admin's tenant row to `subscriptionStatus=active` and emails the customer. Rejected: emails the customer; no plan change.
5. The next login or validation shows the Active Plan card.

> **Edit the status cell once**
>
> The trigger ignores rows that already have *decidedAt*. To correct a mistaken confirmation, edit the Registry and request rows manually (or via Super Admin → Organizations).

7.3 Reminder emails

`runSubscriptionReminders` runs daily at about 08:00 (installed by `installReminderTrigger`). It reads the Registry only and skips suspended organisations. Trials: an email when exactly 2 days remain and another when 0 remain. Active plans: at most one email every 14 days, phrased as "renewal due" when 7 or fewer days remain, otherwise a check-in; the mail suggests Enterprise to Essential customers.

7.4 Capacity monitoring

Google Sheets caps each file at 10,000,000 cells. `checkAllTenantsCapacity` (daily) sums each tenant's *used* range, logs it, projects months remaining to 90% from the recent growth rate, and writes *Healthy* (<70%), *Monitor* (≥70%) or *Action Needed* (≥85%) to the Registry. Crossing into Action Needed emails the Super Admin once. The Super Admin dashboard's **Storage & Capacity** tab displays it; organisation users never see it. The guidance is to archive older records into a separate spreadsheet or split the tenant.

7.5 Suspension

Super Admin → Organizations → Suspend sets Registry status to Suspended. Because every call checks the Registry, all users of that organisation are locked out immediately and login is refused until reactivation. Suspending a single user is done by the organisation Admin.

8. Deployment guide

8.1 Prerequisites

- A Google account that will own the Master spreadsheet, tenant spreadsheets and Drive files (a dedicated account is recommended).
- A static host for the front end (GitHub Pages, Netlify, cPanel, etc.) on HTTPS, with the `app/` folder structure preserved.
- A domain or sub-path you control; decide the final URLs first, because two files embed them.

8.2 Backend

1. Create a new Google Sheet → **Extensions → Apps Script**. Paste `Code.gs`, `MasterRegistry.gs`, `CapacityMonitor.gs`, `Migration.gs` as four script files.
2. Run `setupSheets` (creates PlatformSettings and UpgradeRequests), then `setupMasterRegistry` (creates the Master tabs). Approve the permission prompts (Sheets, Drive, Mail, Triggers).
3. Run `setupSuperAdmin`. **Immediately sign in and change the default password** (Super Admin → My Account).
4. Run `installUpgradeApprovalTrigger`, `installReminderTrigger` and `installCapacityTrigger` once each.
5. Set RESET_PAGE_URL in Code.gs to the public address of `app/reset-password.html`.
6. **Deploy → New deployment → Web app**: Execute as *Me*; Who has access *Anyone*. Copy the `/exec` URL.
7. After *any* later code change, create a **new version** of the deployment (Manage deployments → edit → New version); the one-time setup runs survive redeployment.

8.3 Front end

1. Set API_URL in **both** `okv-auth.js` and `app/okv-auth.js` to the `/exec` URL.
2. Upload the whole folder (not `.gs` files) to the static host, keeping `app/` intact.
3. Update `robots.txt` and `sitemap.xml` with the real domain.
4. Open `app/index.html`, sign in as Super Admin, and complete Branding, Payment Methods, Pricing and (optionally) Communications.
5. Run the walkthrough in the README (sign-up → trial → two isolated organisations → roles → offline/sync → upgrade → super admin).

> **Login fails only in the installed app?**
>
> Check that API_URL was changed in **both** copies of okv-auth.js, and that the service-worker cache version was bumped after editing.

8.4 Configuration reference

Constant / setting	Where	Meaning
SESSION_DAYS (30), RESET_TOKEN_MINUTES (60), TRIAL_DAYS (7)	Code.gs	Lifetimes. Legal pages and marketing copy quote the 7-day trial — keep them in step.
COMPANY_EMAIL / PHONE / WEBSITE	Code.gs	Support contact used in emails (front-end footers hard-code the same values).
PAYMENT_ACCOUNT_*	Code.gs	Default bank account shown until the Super Admin configures payment methods.
PLAN_PRICES, BILLING_CYCLE_DAYS	Code.gs	Fallback prices and period lengths (30 / 182 / 365 days).
COMMS_ROLES	Code.gs and app.html	Roles allowed to send announcements (server and client lists must match).
TENANT_FILES_FOLDER_ID	MasterRegistry.gs	Optional Drive folder for tenant spreadsheets.
CELL_LIMIT, thresholds 0.70 / 0.85 / 0.90	CapacityMonitor.gs	Capacity status boundaries.
SMS / WhatsApp provider	Super Admin → Communications (or Script Properties)	API URL, key, sender ID. Until set, sends report “not configured”.
PlatformSettings JSON	Super Admin dashboard	Branding, payment methods, gateways (stored, not yet wired), plans and prices.

9. Operations and maintenance

9.1 Routine tasks

Frequency	Task
Daily	Check the company inbox for upgrade requests; compare with the bank statement; set Confirmed / Rejected. Glance at Super Admin → Dashboard.
Weekly	Review Storage & Capacity; check the Apps Script *Executions* page for failed runs (reminders, capacity, approvals).
Monthly	Make a copy of the Master spreadsheet and of the tenant folder; review email quota use; rotate the Super Admin password if staff changed.
On every code change	Test, create a new deployment version, bump the service-worker cache version, redeploy static files, and re-run the smoke test.
Before pay-run season (month-end)	Confirm the PAYE/relief logic against current tax guidance (section 11, L1).

9.2 Platform quotas to plan for

Quota	Effect	Mitigation
Apps Script daily email quota	Payslip emailing, reminders and announcements share one sender; large organisations can exhaust it (limits differ for consumer and Workspace accounts).	Use a Workspace account; stagger large sends; move bulk mail to a dedicated provider.
Apps Script execution time and simultaneous executions	Very large organisations or many users syncing at once can be slow or queue.	Keep organisations within the plan sizes; archive old data; consider a proper database for scale.
Sheets 10,000,000 cells per file	Hard cap per tenant.	Capacity monitor; archive historical rows.
Drive storage	Payment screenshots and tenant files count against the owner’s quota.	Clean old screenshots; use a paid storage plan.

9.3 Incident playbook

Symptom	First checks	Action
Everyone gets “Unknown action” or old behaviour	Was a new deployment version created after editing the script?	Redeploy (new version); verify the `/exec` URL is unchanged.
A customer cannot log in but credentials are right	Registry: status, subscriptionStatus, trialEndsAt, subscriptionExpiry.	Reactivate or correct dates via Super Admin → Organizations.
Upgrade marked Confirmed but nothing changed	Is `installUpgradeApprovalTrigger` installed? Is *decidedAt* already filled? Executions log.	Install trigger; clear *decidedAt* and re-set status, or edit the Registry directly.
Emails not arriving	Mail quota; spam folder; recipient address on the record.	Retry later; fix addresses; move to a Workspace sender.
Customer lost archived months	Was the script redeployed with the `payrollHistory` fix? Does a backup exist?	Restore the customer’s JSON backup; ensure the fixed backend is live.
Tenant spreadsheet deleted or inaccessible	Registry *spreadsheetId*; Drive trash.	Restore from trash / version history; update the Registry ID; clear the script cache key.

10. Testing

10.1 Automated checks run for this release

A Playwright/Chromium harness loaded each page with third-party libraries vendored locally and a mock API standing in for Apps Script. Results:

Check	Result
Demo: 21 tabs for Admin, and the exact tab set for each of the 7 roles (compared with the application's own role table)	8 of 8 views pass
Demo: 31 functional checks (chat send, announcement and payslip-email results, subscription form and request list, manuals, add-employee modal, payslip viewer, reports, settings, users, backup download, leave, loans, My Portal, popup, mobile layout)	31 of 31 pass
Real app: every tab for Admin and all 7 roles, with subscription status *trialing* and *active*	0 JavaScript errors
Super Admin: all 9 tabs against a mock API	0 JavaScript errors
Sync regression: archive a month, sync against a backend that (a) omits and (b) supports payrollHistory	Archived months retained in both cases; stored on the server in (b)
Visual: desktop (1400 and 1100 px) and phone (390 px) layouts of the dashboard and subscription tab	No clipping or overflow

What was not tested

The live Apps Script backend (email delivery, triggers, Drive uploads, real spreadsheets) and real payment flows were not exercised. Run the README walkthrough on your deployment before going live.

10.2 Release smoke test (10 minutes)

1. Open the demo; click every tab; switch "View as" through all roles.
2. Sign up a new organisation; confirm welcome email, trial card and install page.
3. Add two employees, enter payroll input, **Confirm & Archive**, press Sync, reload: archived month must persist.
4. Create a Self-Service user linked to an employee; sign in on a second device; verify My Portal.
5. Submit an upgrade request; mark it Confirmed; verify email and Active Plan card.
6. Suspend and reactivate the organisation from Super Admin; verify lock-out and recovery.

11. Known limitations and roadmap

ID	Limitation	Impact	Suggested work
L1	**PAYE logic predates the Nigeria Tax Act 2025** (seven bands + CRA). From 1 Jan 2026 the CRA is abolished, there is a ₦800,000 zero band, six bands to 25% and a Rent Relief.	PAYE on 2026 pay runs will differ from current law. **Highest priority.**	Add a dated tax-profile engine (legacy vs NTA 2025), a rent-relief declaration field per employee, and update the PAYE tab and worked examples.
L2	Plan limits advertised on the pricing page (Essential: 20 employees, 1 Admin + 3 Users) are **not enforced** in the code.	Revenue leakage; unclear expectations.	Enforce in `pushEntity`/`adminCreateUser` with friendly upgrade prompts.
L3	Role checks are client-side only (S1).	Privacy risk if a user bypasses the UI.	Server-side role-scoped `getData`/`pushEntity`.
L4	Life assurance is computed and displayed but not deducted from net pay; overtime and bonus are not included in the taxable figure.	May not match an employer's policy.	Make both configurable per organisation.
L5	Sync replaces whole entity types; concurrent offline edits on different devices lose the earlier sync. No locking on the server.	Possible lost updates in busy teams.	Per-record merge with version stamps; `LockService` around `pushEntity`.
L6	Payment gateways (Paystack, Remita, Flutterwave) are configurable but not integrated; SMS/WhatsApp need a provider.	Manual confirmation workload.	Webhook-based confirmation; provider adapters.
L7	Upgrade approval requires an operator to edit a sheet cell.	Delay and human error.	Approve/reject buttons in the Super Admin dashboard.
L8	Bank payment file, payslip PDF generation and statutory remittance schedules are not produced by the system.	Extra manual steps each month.	Export bank-upload CSV formats; server-side PDF payslips; PAYE/pension schedules.
L9	Apps Script/Sheets performance and quota ceilings (section 9.2).	Limits growth of very large customers.	Plan a migration path to a managed database for large tenants.
L10	Security findings S1–S7.	See section 6.2.	Schedule as a hardening release.

12. Change log for this release

Area	Change
Sync bug (data loss)	Added `payrollHistory` to the backend's allowed entity types; the client no longer replaces local archives when the server omits them. **Requires backend redeploy.**
Dashboard boot bug	Removed a line that destroyed the trial-card plan element and caused every later top-bar refresh to throw; new trial users could be stuck on the loading screen.
KPI cards	Naira figures no longer wrap mid-number in the six-across layout; decorative icons hide on very narrow cards.
Trial countdown	The Upgrade & Subscription tab now rounds days the same way as the dashboard banner.
Super Admin	The Storage & Capacity page now shows its proper title.
Demo	Added Upgrade & Subscription; Messages, Announcements and Manual now run the real code via an in-browser mock API; role table synchronised; "View as" role switcher; trial strip, Upgrade and Sync controls match the real app; dev warning banner hidden.
Service worker	Cache version raised to `okv-payroll-online-v4` so installed apps pick up the fixes.
Documentation	Master Manual, this Full Documentation, Sales & Marketing Pack and Proposal Samples produced.

Also update the README

The README still says Upgrade & Subscription is left out of the demo and mentions a static Messages sample; both are now different. Update those two paragraphs when you next edit it.

Appendix — Quick reference

Apps Script one-time functions

Function	Run when	Effect
setupSheets	Once, new install	Creates PlatformSettings and UpgradeRequests tabs.
setupMasterRegistry	Once, new install	Creates Registry, UserIndex, SessionIndex, ResetIndex, SuperAdmins, CapacityLog.
setupSuperAdmin	Once	Creates the platform Super Admin login (change the password straight away).
installUpgradeApprovalTrigger	Once	On-edit trigger that processes Confirmed/Rejected upgrade rows.
installReminderTrigger	Once	Daily ~08:00 reminder emails.
installCapacityTrigger	Once	Daily tenant capacity check.
migrateToPerTenant / cleanupOldTenantSheets	Only when upgrading an old single-sheet install	Moves each organisation to its own spreadsheet; then clears old data from the Master (manual).

Role table (client and server)

`ROLE_SECTIONS` in app.html (and demo.html) defines tabs per role; `COMMS_ROLES` = HR/Payroll Super Admin, HR Manager, HR Officer, Payroll Officer is duplicated in `Code.gs`. Keep both in step.

Contacts

OKV Technology Consults — technologyokv@gmail.com • +234 810 414 1138 • www.okvtechnology.com